Mid-Decode Model Migration Under a Per-App Memory Limit: Moving a Generation to a Smaller Model Without Holding Both

NagaYu*

Abstract

Prior work first. Closed-form ridge maps between the KV caches of different-size models in one family were introduced by Heo et al. [8]. They apply them at the *prefill hand-off point*. DroidSpeak [9] shares KV caches between models that have the same architecture and are fine-tuned from the same base. This paper adds no new mapping and claims nothing about mapping quality. Its scope is limited to three conditions that this prior work does not address: the switch happens *mid-decode*, between two tokens of one generation; it happens *under memory exhaustion*, triggered by the app's distance to its own per-process limit; and the two models are *never resident together*.

We implement this setting in MLX on Apple silicon and test it against an external jetsam emulator. We compare it with restart, always-small, re-prefill, and the closed-form ridge map of [8] applied mid-decode. That last baseline is reproduced to the letter and runs in the hand-off shape, with both models loaded. Results are **PENDING**. iPhone measurements, energy on device, and human ratings are **PENDING:** not yet collected.

1 Scope and prior work

What is not ours. Mapping one model's KV cache into another model's geometry is prior work. Heo et al. [8] fit, for every target layer and KV head, separate key and value maps in closed form by ridge regression ($\lambda = 0.01$). The input is the concatenated KV heads of the top-k source layers, ranked by single-source R^2. Keys have RoPE [12] removed before the map and re-applied after it. The transfer happens at the *prefill hand-off point*: the receiver gets the sender's prefix cache and decodes from there. Their main results are small-to-large over matched KV geometry. Large-to-small is reported only on HellaSwag, where one pair keeps 37% of standalone accuracy. DroidSpeak [9] reuses KV caches between models that share an architecture and a foundation model. It recomputes selected layers and reuses the rest, at prefill time. Re-reading a transcript with a new model (re-prefill) is a standard alternative, and we treat it as a baseline. Keeping the first tokens because attention concentrates on them follows StreamingLLM [13].

What this paper claims, and only this. We study one situation. An on-device app is in the middle of generating an answer with a model that nearly fills the app's memory budget. Then something else in the app needs memory. On iOS the app is killed once it crosses its per-process limit [2]. The goal is to continue *the same sentence* with a smaller model, with no retracted tokens and no termination. The contribution is restricted to three conditions:

*Code, mappers and raw measurements: `https://github.com/NagaYu/molt`. Draft generated October 17, 2026; device results pending (see Section 7).

1. **Mid-decode.** The switch happens between two tokens of one generation. The receiver takes the sender's cache for every token fed so far and itself feeds the token already shown to the user.

2. **Under memory exhaustion.** The switch is triggered by the app's headroom to its own limit (`os_proc_available_memory` [4]), polled before every token, before the limit is crossed.

3. **Non-co-resident.** Sender and receiver weights are never resident together. We show what this requires on MLX [7] and test that the kernel-tracked peak footprint stays under the cap.

Nothing else in this paper is claimed as new. That includes the mapping, its fitting procedure, RoPE handling, re-prefill and sink tokens.

The question that decides whether there is a claim at all. If the simple method is enough in this setting, molt has nothing to claim. The simple method is the closed-form map of [8] applied at the token boundary. We therefore reproduce it without shortcuts (condition C) and measure where it is and is not enough. Section 6 states plainly where our own hypothesis fails.

2 The setting

The limit an app actually faces. Apple documents `os_proc_available_memory()` as "the current memory limit minus the memory footprint of your app". It returns 0 once the limit is exceeded. The limit can change during the app's life cycle, and the value is advisory and should not be cached [4]. A jetsam event report records the reason for a termination. `per-process-limit` means the app crossed the limit imposed on all apps; `vm-pageshortage` means system-wide pressure [2]. The UIKit memory warning [3] and the dispatch memory-pressure source [5] report *system* memory pressure. We found no documentation that promises either one fires before a `per-process-limit` termination. Some devices honour a larger limit under the `increased-memory-limit` entitlement [1]. The quantity the limit is checked against is the physical footprint. On Apple silicon, MLX tensors sit in Metal buffers that the resident set size barely counts. We measured a 1 GiB MLX array moving RSS by ≈0 MiB and the footprint by ≈1 GiB. Every memory number in this paper is therefore phys_footprint, never RSS.

The event. A generation runs on model S, whose footprint leaves headroom h under the limit. At token n_p, the app's other work starts to need B more memory, arriving in chunks of 64 MiB per token. B exceeds h, so S cannot stay, while the smaller model T plus B fits. A condition survives if its footprint never exceeds the limit. A co-resident switch needs about T's weights plus the mapper in spare memory at the moment it starts. A non-co-resident switch needs only the KV caches and the mapper. The interesting band is

$$\underbrace{\text{chunk} + \text{margin}}_{\text{ordered needs}} \;\le\; h \;<\; \underbrace{w_T + w_{\text{map}} + \text{margin}}_{\text{co-resident needs}},$$

where one schedule survives and the other cannot. Section 5.4 measures whether the band exists and how wide it is for real tiers.

3 Method

3.1 The mapping (prior work, condition C)

We reproduce the recipe of [8] as written.

- *Features.* Per source layer, all KV heads are flattened. Keys have the source RoPE removed; values are used as is.
- *Targets.* The destination's own keys (RoPE removed) and values, per layer and head.
- *Regression.* Centred ridge with $\lambda = 0.01$ and bias $b = \bar{y} - \bar{x}W$. Covariance is accumulated in float32 and solved in float64.
- *Data.* 500 FineWeb-Edu [10] sequences of 1 024 tokens, sub-sampled with stride 4.
- *Source selection.* For each destination layer, the top-k source layers are kept by a fixed ranking: head-averaged single-source R^2, averaged over K and V.

Keys are re-rotated with the destination RoPE at the same absolute positions.

We had to make two interpretations, and we state both. (i) The text writes $X^\top X + \lambda I$ without saying whether $X^\top X$ is normalised. On $\sim$128 K rows an unnormalised Gram matrix would make $\lambda = 0.01$ a no-op, and their reported collapse at $\lambda = 1$ could not happen. We therefore read $X^\top X$ as the centred covariance and also sweep λ (Table 2). (ii) Heo et al. choose k by downstream accuracy. We choose k by held-out prefix-KV negative log-likelihood (NLL), measured on documents disjoint from both the calibration and the evaluation splits.

Implementation checks in `fit.selftest` and `migrate.selftest`: a planted linear map is recovered to $R^2 = 1$; the R^2 computed from sufficient statistics equals the R^2 of actually applying the fitted mapper to within 10^{-7}; and with an identity mapper, transplanting reproduces a native re-prefill to within 1.4×10^{-6}.

3.2 Mid-decode transplant

The decode loop keeps one invariant: the cache holds KV for exactly the tokens fed so far, and `pending` is the token already shown to the user but not yet fed. At a switch the receiver gets the mapped cache and feeds `pending` itself, so no shown token is retracted. Molt adds two optional *anchors*, which we treat as engineering choices and not as contributions.

- The first s positions are replaced by the receiver's native KV. This is exact, because causal positions depend only on earlier tokens; it follows the sink observation of [13].
- The last t positions are re-fed natively over the mapped history.

Setting $t =$ all gives re-prefill (condition A′). s and $t \leq 32$ are chosen on held-out text only.

3.3 The memory order (the claim)

Releasing a model on MLX is not one call. We measured three facts (Appendix A). (1) After `del`, buffers stay in MLX's allocator cache and are not reused for arrays of other sizes. (2) `mx.clear_cache()` right after `del` can free nothing, because a buffer returns to the cache only after the GPU has finished the last command buffer that used it. (3) After synchronising and clearing, the OS still reclaims the memory asynchronously, in about 10^2 ms. The *ordered* schedule therefore runs as follows:

1. snapshot the source KV;
2. drop the source weights, synchronise, clear, and poll phys_footprint until it has fallen by 90% of the source weight bytes;
3. load the mapper and map;
4. drop the mapper and source KV the same way;
5. load the receiver;
6. apply the anchors and resume.

Table 5 lists the three alternatives we measure against it. *colocated* is the prefill hand-off shape. *del-only* is the natural "drop it, then load". *clear-no-wait* is the call a developer would write.

The trigger. Before every token the loop reads the headroom, which is `os_proc_available_memory()` on iOS and cap − phys_footprint on the Mac. It migrates when the headroom falls below what *the chosen schedule* needs plus the next demand chunk. Each schedule thus gets its own best trigger, and a co-resident schedule is never killed merely for reacting late.

4 Experimental setup

Models. We use Qwen2.5 Instruct [11] in the 4-bit MLX builds an app would ship. The pairs, large to small (the direction memory pressure forces), are 3B→1.5B (matched KV geometry: 2 KV heads, $d_h = 128$) and 1.5B→0.5B (mismatched: d_h 128→64). Mappers are fitted on these exact 4-bit weights. All three sizes share one tokenizer, which we checked by vocabulary hash. Decoding is greedy, so conditions differ only in what the switch did.

Conditions. All conditions run on one machine. Each run is a separate process, so no condition inherits another's allocator state.

- **A** restart: discard the generation and start over on the small model.
- **A′** re-prefill: release, load small, and re-read the prompt plus everything shown so far. This baseline is added because it is the obvious alternative; it is not one of the four named in the brief.
- **B** always small.
- **C** the closed-form map of [8] applied at the token boundary, with the selected k^*, in hand-off shape (colocated) and without anchors.
- **D** molt: the same mapper, the ordered schedule and the held-out-selected anchors.

Ablations change one factor each: C+ordered isolates the schedule, D+colocated the anchors, D with the depth-linear mapper the mapper. D with del-only and D with clear-no-wait test the naive freeing variants.

Jetsam emulation. A separate C process polls the worker's `proc_pid_rusage` at about 1 kHz. It records the instantaneous phys_footprint and the kernel ledger's lifetime maximum (`ri_lifetime_max_phys_footp`) and sends `SIGKILL` the moment either exceeds the cap. The lifetime maximum cannot miss a spike between samples. Phase labels written by the worker attribute every sample to a phase.

Table 1: Every fitted mapper. Held-out = documents disjoint from calibration and evaluation.
PENDING

Table 2: Ridge λ sweep at the selected k^* (centred-covariance reading).
PENDING

Experiments. *I (intrinsic)*: teacher-forced NLL of 128 held-out tokens after the switch, on the FineWeb-Edu evaluation split, at boundary positions $P \in \{32, 128, 256, 512, 768\}$. *Q (quality)*: no cap, with the switch at a fixed token per item. The suite has GSM8K [6] (switch at token 32, mid-reasoning), a needle task with 14 similar records (switch right before the answer value), and open-ended explanations (switch at token 48). *M (memory)*: a cap of $S+h$, where S is the measured footprint of the big model decoding, swept over h; demand arrives as in Section 2.

Metrics.

- **Survival**: the run was not killed.
- **Lost tokens**: tokens shown and then retracted.
- **Visible seam**: the jump in judge NLL across the boundary, $\overline{\text{NLL}}[b, b{+}16) - \overline{\text{NLL}}[b{-}16, b)$, reported relative to A′; and human ratings, which are pending.
- **Continuation quality**: task accuracy, with a killed run scoring 0 and never dropped, plus judge top-1 agreement after the boundary. The judge is the pair's large model.
- **Time to resume**: from trigger to the next shown token. For A, the time until new content appears.
- **Peak footprint during migration**: kernel lifetime maximum.
- **Energy**: CPU energy per process (`ri_energy_nj`) and GPU energy system-wide from IOReport minus an idle baseline (Appendix B).

Throughput is recorded but is not a primary metric. Every aggregate carries a 95% bootstrap interval, paired by item where two conditions are compared.

Platform. Reported per run in the result files: Apple M2, 16 GB, macOS 27.0.1, MLX 0.31.2, mlx-lm 0.31.3. Other users' processes were running during the measurements and swap was nearly full (Section 7). Timings are reported but should be read with that in mind; footprint and survival are not affected.

5 Results

5.1 The prior mapping, reproduced

Figure 1 and Table 1 cover every fitted mapper. **PENDING:** measurements for this subsection.

5.2 Transfer cost against switch position

PENDING: measurements for this subsection.

Table 3: Experiment Q. Accuracy counts every item; jump = judge-NLL jump at the switch (16 tokens each side).

PENDING

Table 4: Experiment M: every cell. Peak = kernel lifetime-max phys_footprint (MiB).

PENDING

5.3 Quality when the switch lands mid-decode

PENDING: measurements for this subsection.

5.4 Survival under a per-process limit

PENDING: measurements for this subsection.

6 Where the hypothesis fails, stated plainly

PENDING: written once all measurements are in.

7 Limitations

- **No iPhone results yet.** Every number in this draft comes from an M2 Mac with a jetsam *emulator*. The device protocol is written and its platform core is self-tested on macOS (`ios/`). It covers the Memory Budget Test that reproduces real kills, Instruments traces, and at least three device generations. The app itself has not been compiled or run, because the build machine's Xcode license is not accepted and no device is attached. The simulator is not used: it enforces no per-app limit. Real limits, the `increased-memory-limit` entitlement, and iOS's own reclaim timing could all move the band of Section 5.4.
- **A shared, overloaded machine.** Other users' processes kept swap nearly full throughout. Footprint, survival and quality are unaffected. Times and GPU energy are inflated and noisy, and are reported with the machine's load average.
- **Energy is partial.** CPU energy is per process. GPU energy is system-wide minus an idle baseline. DRAM energy is missing. A `powermetrics` cross-check is pending.
- **Human ratings are pending.** The seam is measured only with a model judge, which is the pair's own large model.
- **Scope.** One model family (Qwen2.5), 4-bit weights, greedy decoding, English text.

8 Conclusion

PENDING: written once all measurements are in.

Figure 1: Held-out prefix-KV NLL against the number of source layers k used by the closed-form mapper of [8], with 95% CIs. Dashed: the receiver reading the same prefix natively (what re-prefill achieves). Dash-dot: the sender reading it natively.

References

[1] Apple Inc. com.apple.developer.kernel.increased-memory-limit. Apple Developer Documentation, `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit`, . Accessed 2026-10-17.

[2] Apple Inc. Identifying high-memory use with jetsam event reports. Apple Developer Documentation, `https://developer.apple.com/documentation/xcode/identifying-high-memory-use-with-jetsam-event-reports`, . Accessed 2026-10-17.

[3] Apple Inc. didreceivememorywarningnotification. Apple Developer Documentation, `https://developer.apple.com/documentation/uikit/uiapplication/didreceivememorywarningnotification`, . Accessed 2026-10-17.

[4] Apple Inc. os_proc_available_memory. Apple Developer Documentation, `https://developer.apple.com/documentation/os/os_proc_available_memory`, . Accessed 2026-10-17.

[5] Apple Inc. makememorypressuresource(eventmask:queue:). Apple Developer Documentation, `https://developer.apple.com/documentation/dispatch/dispatchsource/makememorypressuresource(eventmask:queue:)`, . Accessed 2026-10-17.

Figure 2: Experiment I. Extra NLL of the receiver decoding from a transferred cache, relative to reading the same text natively, as a function of how much context precedes the switch. Bands are paired 95% CIs over evaluation documents.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[8] Taekyung Heo, Rasoul Shafipour, Ritchie Zhao, Maximilian Golub, Mohammad Mahdi Kamani, Ritika Borkar, Makesh Tarun Chandran, Pantea Zardoshti, and Bita Darvish Rouhani. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026.

[9] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[10] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Figure 3: Experiment Q. GSM8K and needle exact match, and judge top-1 agreement after the switch, per condition. Error bars are 95% bootstrap CIs.

Table 5: Memory schedules. All of them compute the same transfer; they differ only in what is resident when.

schedule	order	resident at peak
colocated	load receiver, map, drop sender	sender + receiver + mapper
del-only	drop sender refs, map, load receiver	sender (cached by MLX) + receiver
clear-no-wait	drop, `clear_cache()`, load at once	depends on reclaim timing
ordered	drop, sync, clear, *wait*, map, drop mapper, load	max(sender, KV + mapper, receiver)

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[13] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

A What freeing a model costs on MLX

All three facts below are measured by `mlx/molt_mlx/footprint.py::selftest` on every run of the test suite.

1. A 256 MiB MLX array adds 262 MiB to phys_footprint, while total RSS is 32 MiB.
2. After `del` the footprint is unchanged (+262 MiB). `mx.clear_cache()` immediately after `del`, followed by a 300 ms sleep, freed the buffer in only 1 of 5 runs; in the others the footprint stayed up by as much as +262 MiB. Whether it frees depends on whether the GPU has finished with the buffer, so it is not something to rely on.
3. `mx.synchronize()`, then `clear_cache()`, then polling returned the memory every time, with a median of 163 ms (range 0–257 ms).

The peak-footprint test (`mlx/tests/test_peak_footprint.py`) runs each schedule in its own process under a cap of (source-resident footprint + 40 MiB). The models are a synthetic 290 MiB source and a 109 MiB destination, so the cap is below source plus destination. Ordered peaks at 694 MiB under a 734 MiB cap; colocated and del-only are killed in `load_dst`. clear-no-wait survived 3 of 3 runs at this size, so it is *not* shown to fail there; the full-size result is in Section 5.4.

Figure 4: Experiment M. Survival rate against headroom h over the big model's own footprint, one process per run under the jetsam emulator. The dotted line marks the headroom a co-resident switch needs (receiver weights + mapper + margin).

B Energy without root

`ri_energy_nj` counts CPU energy only: a busy MLX matmul loop reads 0.03 W in it. IOReport's "GPU Energy" channel reads 12.15 W for the same loop, with no root required. Energy is reported as these two named parts and never summed silently. DRAM energy is unavailable without `powermetrics`; `bench/powermetrics.sh` is the root cross-check.

C Reproducing every number

```
python -m pytest mlx/tests --import-mode=importlib     # every module's selftest + peak test
python -m bench.fit_mappers --pair 1.5b-0.5b           # fit, sweep, select (held-out only)
python -m bench.intrinsic   --pair 1.5b-0.5b           # experiment I
python -m bench.run_quality --pair 1.5b-0.5b           # experiment Q (one process per shard)
python -m bench.run_memory  --pair 1.5b-0.5b           # experiment M (one process per run)
python -m bench.score       --pair 1.5b-0.5b           # judge scores
python -m bench.analyze && python figures/make_figures.py   # summary, figures, this paper's ma
cd ios && swift run MoltCoreSelfTest                   # Swift core
```

Figures and every number in the text are produced by the last command from `bench/results`; none is typed by hand.